

Software Testing

HOW TO WRITE TEST CASES

What Is a Test Case in Software?

A test case measure functionality across a set of actions or conditions to verify the expected result. They apply to any software application, can use manual testing or an automated test, and can make use of test case management tools.

Different Types of Test Cases

Some common test case examples would be the following:

Test Case Type	Description	Test Step	Expected Result	Status
Functionality	Area should accommodate up to 20 characters	Input up to 20 characters	All 20 characters in the request should be appropriate	Pass or Fail
Security	Verify password rules are working	Create a new password in accordance with rules	The user's password will be accepted if it adheres to the rules	Pass or Fail
Usability	Ensure all links are working properly	Have users click on various links on the page	Links will take users to another web page according to the on-page URL	Pass or Fail

How to Write Software Test Cases

Every test case can be broken down into 8 basic steps.

Step 1: Test Case ID

Test cases should all bear unique IDs to represent them.

Step 2: Test Description

This description should detail what unit, feature, or function is being tested or what is being verified.

Step 3: Assumptions and Pre-Conditions

This entails any conditions to be met before test case execution. One example would be requiring a valid account for a login.

How to Write Software Test Cases

Step 4: Test Data

This relates to the variables and their values in the test case. In the example of an email login, it would be the username and password for the account.

Step 5: Steps to be Executed

These should be easily repeatable steps as executed from the end user's perspective. For instance, a test case for logging into an email server might include these steps:

1. Open email server web page.
2. Enter username.
3. Enter password.
4. Click "Enter" or "Login" button.

How to Write Software Test Cases

Step 6: Expected Result

This indicates the result expected after the test case step execution. Upon entering the right login information, the expected result would be a successful login.

Step 7: Actual Result and Post-Conditions

As compared to the expected result, we can determine the status of the test case. In the case of the email login, the user would either be successfully logged in or not. The post-condition is what happens as a result of the step execution such as being redirected to the email inbox.

Step 8: Pass/Fail

Determining the pass/fail status depends on how the expected result and the actual result compare to each other.

CSSE3113 – Introduction to Software Engineering

Software Testing

Software Testing

The process of operating a system or component under specified conditions, observing or recording the results, and making an evaluation of some aspect of the system or component.

Software Testing

Testing can be defined as

“A process of analyzing a software item to detect the differences between existing and required conditions and to evaluate the features of the software item”

According to IEEE standard

Software Testing

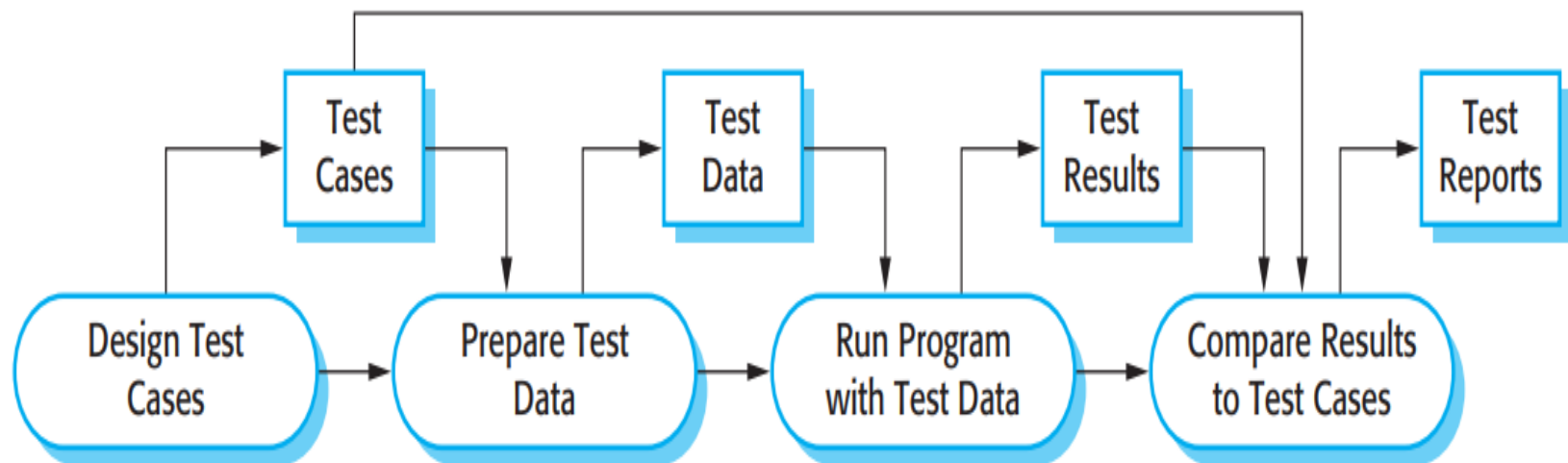
- Testing is intended to show that a program does what it is intended to do and to discover program defects before it is put into use.
- When you test software, you execute a program using artificial data. You check the results of the test run for errors, anomalies, or information about the program's non-functional attributes.

Who Does Testing?

In most cases, following professionals are involved in testing of a system within their respective capacities:

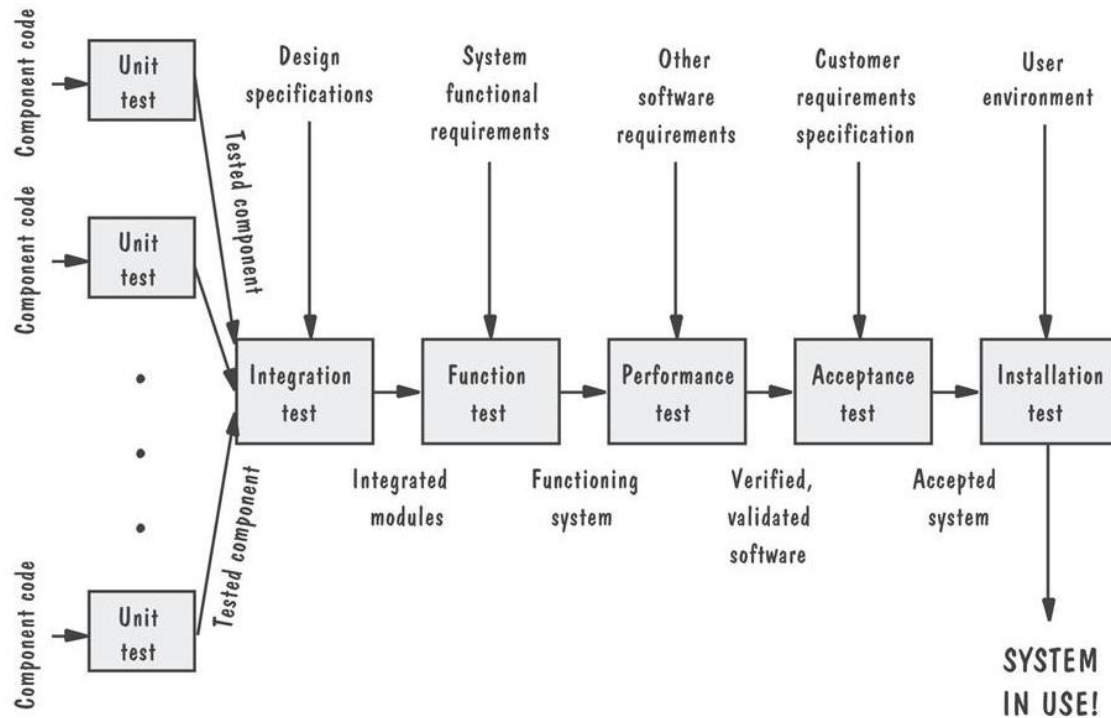
- Software Tester
- Software Developer
- Project Leader/Manager
- End User

A model of the software testing process



Levels of Testing

Testing Organization Illustrated



Types of Testing

Manual testing

- This type includes the testing of the Software manually i.e. without using any automated tool.
- There are different stages for manual testing like unit testing, Integration testing, System testing and User Acceptance testing.
- Testers use test plan, test cases or test scenarios to test the Software to ensure the completeness of testing.

Automation Testing

- Automation testing which is also known as *Test Automation*, is when the tester uses another software to test the software.
- This process involves automation of a manual process.
- Integration of computerized tools into the process of software testing.
- Code auditing
- Coverage monitoring
- Load tests...etc

Software Testing

Typically, a commercial software system has to go through three stages of testing:

Development testing, where the system is tested during development to discover bugs and defects. System designers and programmers are likely to be involved in the testing process.

Release testing, where a separate testing team tests a complete version of the system before it is released to users. The aim of release testing is to check that the **system meets the requirements of system stakeholders.**

User testing, where users or potential users of a system test the system in their own environment.

For software products, the 'user' may be an internal marketing group who decide if the software can be marketed, released, and sold. Acceptance testing is one type of user testing.

Classification According to Testing Concept

- Black Box Testing
- White Box Testing
- Gray Box Testing

Black Box Testing

- The technique of testing without having any knowledge of the interior workings of the application is Black Box testing.
- The tester does not have access to the source code.
- A tester will interact with the system's user interface by providing inputs and examining outputs without knowing how and where the inputs are worked upon.

Black Box Testing

Advantages

- Well suited and efficient for large code segments.
- Code Access not required.
- Clearly separates user's perspective from the developer's perspective.
- Large numbers of moderately skilled testers can test the application with no knowledge of implementation, programming language or operating systems.

Black Box Testing

Disadvantages

- Inefficient testing, due to the fact that the tester only has limited knowledge about an application.
- Blind Coverage, since the tester cannot target specific code segments or error prone areas.
- The test cases are difficult to design.

White Box Testing

- White box testing is the detailed investigation of internal logic and structure of the code.
- White box testing is also called glass testing or open box testing.
- In order to perform white box testing on an application, the tester needs to have knowledge of the internal working of the code.
- The tester needs to have a look inside the source code and find out which unit/chunk of the code is behaving inappropriately.

White Box Testing

Advantages

- As the tester has knowledge of the source code, it becomes very easy to find out which type of data can help in testing the application effectively.
- Extra lines of code can be removed which can bring in hidden defects.
- Due to the tester's knowledge about the code, maximum coverage is attained during test scenario writing.

White Box Testing

Disadvantages

- Due to the fact that a skilled tester is needed to perform white box testing, the costs are increased.
- Sometimes it is impossible to look into every nook and corner to find out hidden errors.
- It is difficult to maintain white box testing as the use of specialized tools like code analyzers and debugging tools are required.

Black Box vs White Box

Black Box Testing	White Box Testing
The Internal Workings of an application are not required to be known	Tester has full knowledge of the Internal workings of the application
Also known as closed box testing, data driven testing and functional testing	Also known as clear box testing, structural testing or code based testing
Performed by end users and also by testers and developers	Normally done by testers and developers
Testing is based on external expectations - Internal behavior of the application is unknown	Internal workings are fully known and the tester can design test data accordingly
This is the least time consuming and exhaustive	The most exhaustive and time consuming type of testing
Not suited to algorithm testing	Suited for algorithm testing

Gray Box Testing

- › Gray box testing is a software testing technique that uses a combination of black box testing and white box testing. Gray box testing is not black box testing, because the tester does know some of the internal workings of the software under test.
- › In gray box testing, the tester applies a limited number of test cases to the internal workings of the software under test. In the remaining part of the gray box testing, one takes a black box approach in applying inputs to the software under test and observing the outputs.
- › This is particularly important when conducting **integration testing** between two modules of code written by two different developers, where only the **interfaces are exposed for test**.



Testing strategies/ Development testing

- There are basically two testing strategies:

Big bang testing: tests the software as a whole, once the completed package is available.

Incremental testing:

- Tests the software piecemeal – software modules are tested as they are completed (unit tests)
- followed by groups of modules composed of tested modules integrated with newly completed modules (integration tests).
- Once the entire package is completed, it is tested as a whole (system test).

Unit Testing

- Unit testing is performed by the respective developers on the individual units of source code assigned areas.
- The goal of unit testing is to isolate each part of the program and show that individual parts are correct in terms of requirements and functionality.

Integration Testing /System testing

The testing of combined parts of an application to determine if they function correctly together is Integration testing.

System Testing

- This is the next level in the testing and tests the system as a whole.
- Once all the components are integrated, the application as a whole is tested to see that it meets Quality Standards.
- This type of testing is performed by a specialized testing team.

Regression Testing

- Whenever a change in a software application is made it is quite possible that other areas within the application have been affected by this change.
- The intent of Regression testing is to ensure that a change, such as a bug fix did not result in another fault being uncovered in the application.

User Testing

In practice, there are three different types of user testing:

Alpha testing, where users of the software work with the development team to test the software at the developer's site.

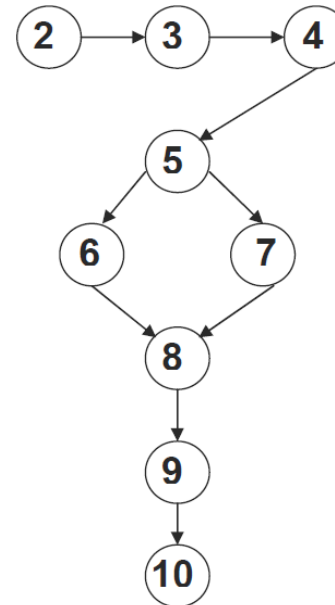
Beta testing, where a release of the software is made available to users to allow them to experiment and to raise problems that they discover with the system developers.

Acceptance testing, where customers test a system to decide whether or not it is ready to be accepted from the system developers and deployed in the customer environment.

Whitebox Testing

Control Flow Graph

1. Program 'Simple Subtraction'
2. Input (x, y)
3. Output (x)
4. Output (y)
5. If $x > y$ then DO
6. $x - y = z$
7. Else $y - x = z$
8. EndIf
9. Output (z)
10. Output "End Program"



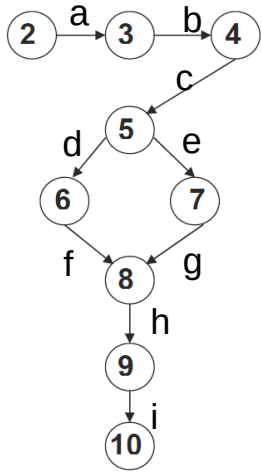
Cyclomatic Complexity is defined as:

$CC = E - N + 2P$ Where:

- **E** = Number of edges (transitions or connections between nodes in the flow graph)
- **N** = Number of nodes (decision points or statements in the flow graph)
- **P** = Number of connected components or exits (typically $P=1$ for a single program)

Cyclomatic Complexity = $E - N + 2 = 9 - 9 + 2 = 2$

Whitebox Testing graph (Adjacency) matrices



Control Flow Graph

1. Program 'Simple Subtraction'
2. Input (x, y)
3. Output (x)
4. Output (y)
5. If $x > y$ then DO
6. $x - y = z$
7. Else $y - x = z$
8. EndIf
9. Output (z)
10. Output "End Program"

		Connected to Node								
		2	3	4	5	6	7	8	9	10
Node	2		a							
	3			b						
	4				c					
	5					d	e			
	6							f		
	7							g		
	8								h	
	9									i
	10									
	1									
	0									

Graph Matrix

To this point, the graph matrix is nothing more than a tabular representation of a flow graph.

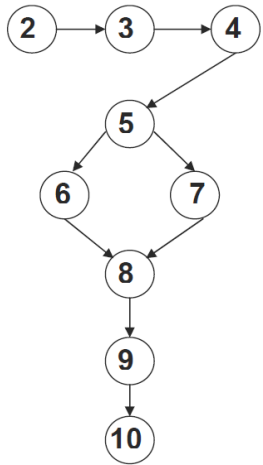
Whitebox Testing

graph (Adjacency) matrices

- By adding a link weight to each matrix entry, the graph matrix can become a powerful tool for evaluating program control structure during testing.
- The link weight provides additional information about control flow. In its simplest form, the link weight is 1 (a connection exists) or 0 (a connection does not exist).
- Following are the steps to compute the cyclomatic complexity in Graph Matrices :
 1. Count the number of 1s in each row and write it in the end of the row
 2. Subtract 1 from this count for each row (ignore the row if its count is 0)
 3. Add the count of each row calculated previously
 4. Add 1 to this total count
 5. The final sum in Step 4 is the cyclomatic complexity of the control flow graph

Whitebox Testing

graph (Adjacency) matrices



Cyclomatic Complexity (Control Flow graph)

$$\begin{aligned} &= E - N + 2 \\ &= 9 - 9 + 2 \\ &= 2 \end{aligned}$$

1. Program 'Simple Subtraction'
2. Input (x, y)
3. Output (x)
4. Output (y)
5. If $x > y$ then DO
6. $x - y = z$
7. Else $y - x = z$
8. EndIf
9. Output (z)
10. Output "End Program"

Sum of column =1

Cyclomatic Complexity = 1+1=2
(Graph Matrix)

[illegible]

Cyclomatic Complexity

Cyclomatic Complexity	Meaning
1 – 10	•Structured and Well Written Code •High Testability •Less Cost and Effort
10 – 20	•Complex Code •Medium Testability •Medium Cost and Effort
20 – 40	•Very Complex Code •Low Testability •High Cost and Effort
> 40	•Highly Complex Code •Not at all Testable •Very High Cost and Effort

Rules

- We will not compute CC for whole project but for per function or per method.
- Compute CC for each function separately.
- In $CC = E - N + 2P$:
- P is number of **connected components** (normally 1 per function)

Example

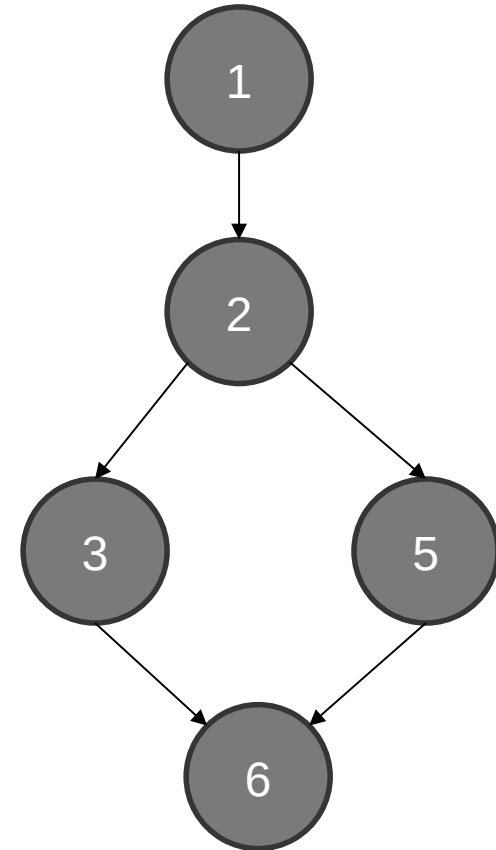
```
1. int max(int a, int b) {  
2.     if (a > b)  
3.         return a;  
4.     else  
5.         return b;  
6. }
```

E=5

N=5

P=1

$CC = 5 - 5 + 2(1) \Rightarrow 2$



Example with loop + condition

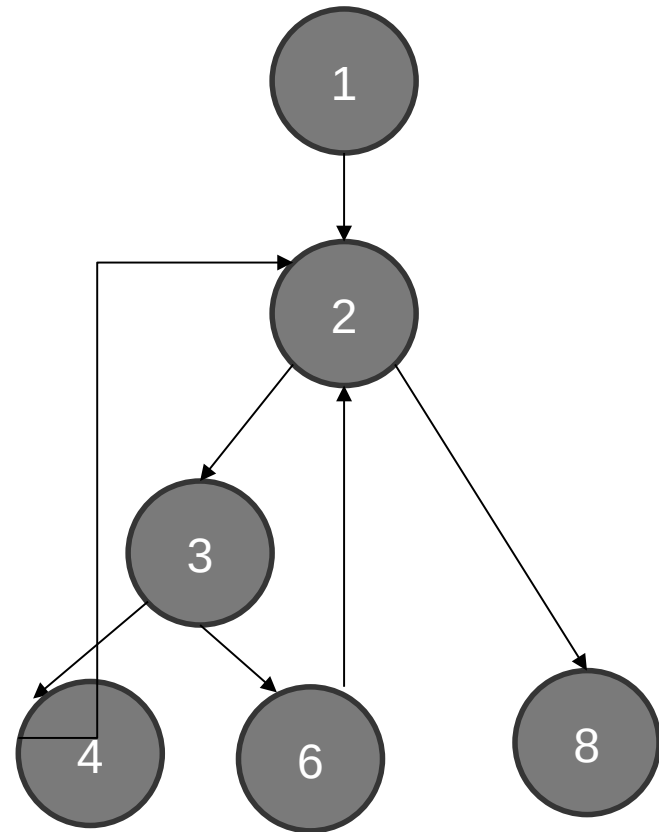
```
1. void fun(int x) {  
2.     while (x > 0) {  
3.         if (x % 2 == 0)  
4.             x--;  
5.         else  
6.             x = x - 2;  
7.     }  
8. }
```

E=7

N=6

P=1

CC=7-6+2(1)=>3



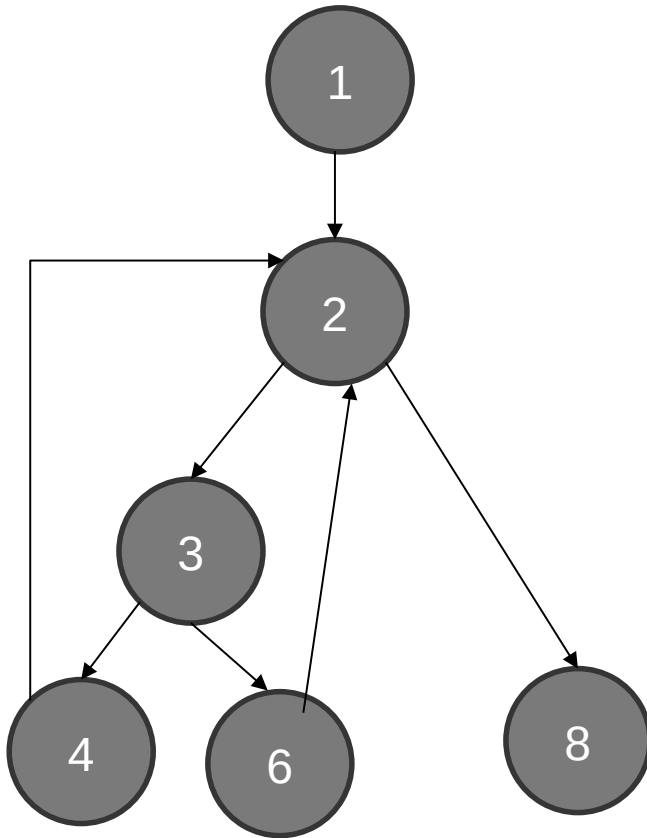
Code Example with Multiple Independent Functions

```
1. void funcA(int x) {  
2.     while (x > 0) {  
3.         if (x % 2 == 0)  
4.             x--;  
5.         else  
6.             x = x - 3;  
7.     }  
8. }  
  
9. void funcB(int y) {  
10.    for (int i = 0; i < y; i++) {  
11.        if (i == 5)  
12.            break;  
13.    }  
14. }
```

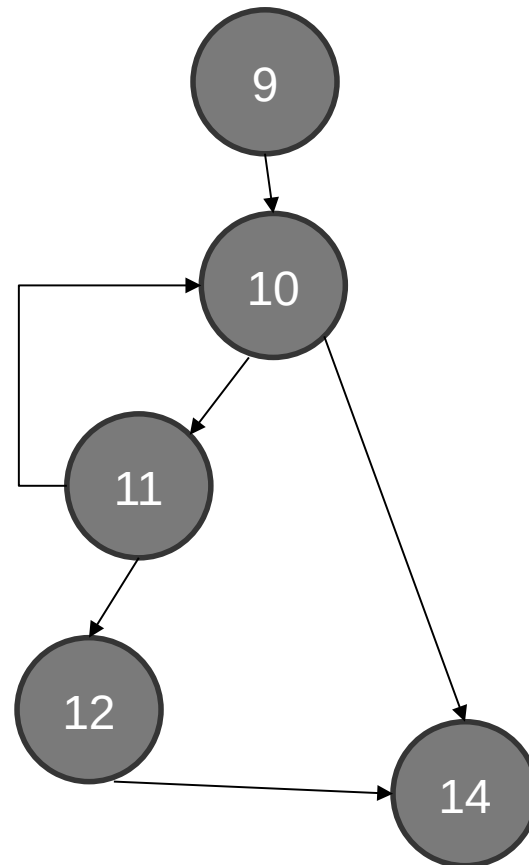
- funcA and funcB are independent
- No function calls between them
- When combined into one flow graph, they form two disconnected components
- So $P=2$ not 1

Code Example with Multiple Independent Functions

1. $E=7$ $N=6$



• $E=6$ $N=5$



Code Example with Multiple Independent Functions

- So $E=7+6=13$
- $N=6+5=11$
- $P=2$
- $CC=13-11+2(2)$
- $=2+4$
- $=6$

In a class

```
class Account {  
    int balance;  
    boolean isPositive() {  
        if (balance > 0)  
            return true;  
        return false;  
    }  
}
```

- Nodes:

1. Start
2. if (balance > 0) → decision node
3. return true
4. return false
5. End

$$\begin{aligned} CC &= E - N + 2P = 5 - 5 \\ &+ 2 \\ &= 2 \end{aligned}$$

In a class

```
class Demo {  
  
    void A(int x) {  
        if (x > 0)  
            x++;  
    }  
  
    void B(int y) {  
        for (int i = 0; i < y; i++) {  
            if (i == 5)  
                break;  
        }  
    }  
}
```

A and B will be treated independently as we discussed previously and $P=2$

Nested functions

```
1. def outer(x):
2.     if x > 0:
3.         def inner(y):
4.             while y > 0:
5.                 y -= 1
6.                 inner(x)
7.     else:
8.         print("x is non-
           positive")
```

- P=2 why?
- 1-2
- 2-6(if true)
- 2-8(if false)
- 6-end of outer
- 8-end of outer

E=5 N=5

Now other fun

- 3-4
- 4-5(if true)
- 5-4 (for loop)
- 4-end of inner

E=4 N=4

CC=9-9+4=4

In exception handling

- **try** itself does not create a separate node, unless it contains executable statements.
- **catch** counts as a decision point / separate node because it introduces an alternative control flow.
- **finally**, is usually treated as a normal node since it always executes.

```
1. void process() {  
2.     try {  
3.         action1();  
4.     } catch(Exception e) {  
5.         action2();  
6.     } finally {  
7.         cleanup();  
8.     }  
9. }
```

Node	Meaning
1	Start
2	action1() in try
3	catch block → action2()
4	finally → cleanup()
5	End

- 1-2
- 2-3(if any issue)
- 2-4(in any case will execute)
- 3-4
- 4-5

$E = 5, N = 5, P = 1$

If any recursive function call

```

1.  def factorial(n):
2.      if n <= 1:
3.          return 1
4.      else:
5.          return n * factorial(n-1)

```

loop executes multiple times in the same activation

Each recursive call is **a new activation**, not part of the current function's control flow and CFG **models one function activation**.
So no edge between 4 and 1 like loops

Node	Meaning
1	Start
2	If true
3	return
4	Recursive call
5	End

- 1-2
- 2-3(if true)
- 2-4
- 3-5
- 4-5

$$E = 5, N = 5, P = 1$$

Practice

Cyclomatic Complexity Control Flow Graph & Graph / Adjacency Matrix

1. IF A = 10 THEN
2. IF B > C THEN
3. A = B
4. ELSE A = C
5. ENDIF
6. ENDIF
7. Print A
8. Print B
9. Print C